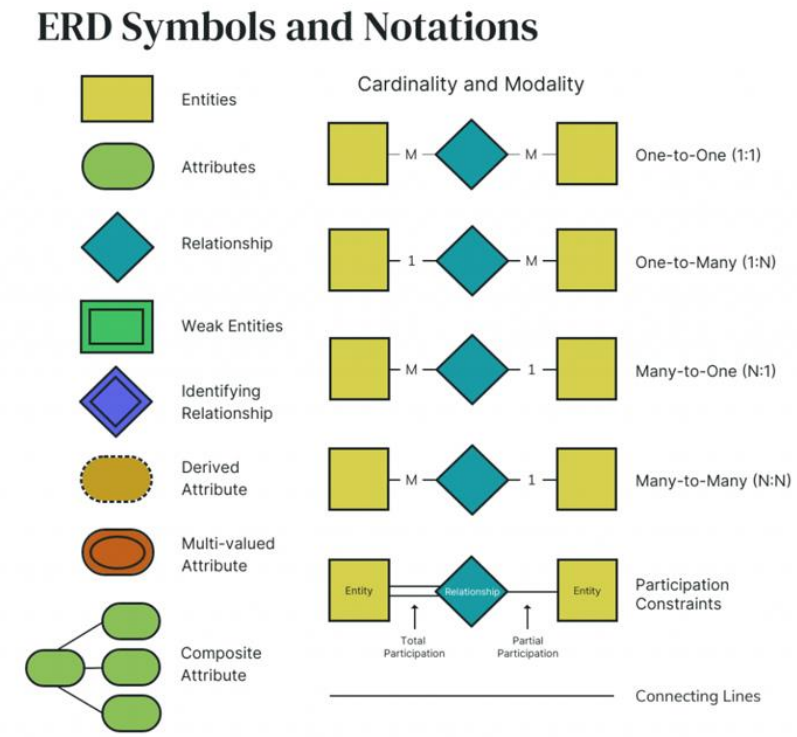
SQL Reference

Command	Description	Syntax	Example
CREATE / ALTER TABLE	Creates a table, or adds a column to an existing table	CREATE TABLE t (col type, ...); ALTER TABLE t ADD COLUMN col type;	CREATE TABLE s (id SERIAL PRIMARY KEY, name VARCHAR(50) NOT NULL);
Data types	INT; SERIAL (auto-incrementing integer); NUMERIC(p,s) exact decimal; VARCHAR(n); TEXT; DATE; BOOLEAN	col_name type	price NUMERIC(8,2)
Constraints	PRIMARY KEY (also composite), NOT NULL, DEFAULT, FOREIGN KEY	col type REFERENCES t(col) PRIMARY KEY (a, b) col type DEFAULT value	dept_id INT REFERENCES dept(dept_id); PRIMARY KEY (s_id, c_id)
INSERT / UPDATE / DELETE	Adds rows, changes values, removes rows (always use WHERE with UPDATE and DELETE)	INSERT INTO t (c1, c2) VALUES (v1, v2); UPDATE t SET c = expr WHERE cond; DELETE FROM t WHERE cond;	UPDATE s SET age = age + 1 WHERE id = 3;
SELECT / DISTINCT / AS / LIMIT	Chooses columns, removes duplicate rows, renames an output column, limits the number of rows	SELECT DISTINCT c AS alias FROM t LIMIT n;	SELECT DISTINCT age AS a FROM s LIMIT 5;
WHERE operators	Filters rows: =, <>, <, >, AND, OR, NOT, IS NULL, BETWEEN (inclusive), IN, LIKE (% = any text, _ = one character)	WHERE c BETWEEN a AND b WHERE c IN (v1, v2) WHERE c LIKE 'A%'	WHERE age BETWEEN 20 AND 25 AND email IS NOT NULL
ORDER BY	Sorts the result; ASC is the default; NULLS FIRST / NULLS LAST places NULL values	ORDER BY c1 DESC, c2 ASC	ORDER BY age DESC, name
INNER JOIN / LEFT JOIN	INNER keeps matching rows only; LEFT keeps every row of the left table (NULL when there is no match)	FROM a JOIN b ON a.k = b.k FROM a LEFT JOIN b ON a.k = b.k	FROM s LEFT JOIN c ON s.id = c.s_id WHERE c.s_id IS NULL
Self join	Joins a table to itself using two aliases	FROM t x JOIN t y ON x.fk = y.pk	FROM s a JOIN s b ON a.mentor_id = b.id
Subquery / EXISTS	A query inside parentheses; EXISTS is true when the subquery returns at least one row	WHERE c > (SELECT AVG(c) FROM t) WHERE EXISTS (SELECT 1 FROM t2 WHERE t2.k = t1.k)	WHERE age > (SELECT AVG(age) FROM s)
WITH (CTE)	A temporary named result that can be used inside one query	WITH name AS (SELECT ...) SELECT ... FROM name;	WITH a AS (SELECT dept, AVG(age) AS av FROM s GROUP BY dept) SELECT * FROM a;
UNION / INTERSECT / EXCEPT	Combines two queries that have the same columns: all distinct rows / rows in both / rows of the first that are not in the second	q1 UNION q2 q1 INTERSECT q2 q1 EXCEPT q2	SELECT name FROM s EXCEPT SELECT name FROM t

Command	Description	Syntax	Example
COUNT / SUM / AVG / MIN / MAX / ROUND	Aggregate functions (they ignore NULL, COUNT(*) counts rows); ROUND(x, n) rounds to n decimals	COUNT(*), AVG(c), ROUND(AVG(c), 1)	SELECT ROUND(AVG(age), 1) FROM s;
GROUP BY / HAVING	Groups rows for aggregates; WHERE filters rows before grouping, HAVING filters groups after it	SELECT c, COUNT(*) FROM t WHERE cond GROUP BY c HAVING COUNT(*) > 1	SELECT dept, COUNT(*) FROM s GROUP BY dept HAVING COUNT(*) > 2;
OVER / PARTITION BY	Window function: calculates across related rows without collapsing them	func() OVER (PARTITION BY c1 ORDER BY c2)	AVG(age) OVER (PARTITION BY dept)
ROW_NUMBER / RANK / DENSE_RANK	Unique number / rank with gaps after ties (1, 1, 3) / rank without gaps (1, 1, 2)	RANK() OVER (ORDER BY c DESC)	ROW_NUMBER() OVER (ORDER BY age DESC)

ER Notation



Question 1: Company Database (50 points)

Use PostgreSQL syntax. The database has three tables (PK = primary key, FK = foreign key):

```
departments(dept_id PK, dept_name, city)
employees(emp_id PK, name, age, email, salary, dept_id FK, manager_id FK, hire_date)
contractors(contractor_id PK, name, hourly_rate)
```

employees.dept_id refers to departments.dept_id, and employees.manager_id refers to employees.emp_id.

The tables contain this data:

```
INSERT INTO departments VALUES
  (10, 'Sales', 'Baku'), (20, 'IT', 'Baku'), (30, 'HR', 'Ganja'),
  (40, 'Finance', 'Sumgait'), (50, 'Legal', 'Ganja');

INSERT INTO employees VALUES
  (1, 'Aysel', 45, 'aysel@corp.az', 5000, 20, NULL, '2015-03-01'),
  (2, 'Rauf',  38, 'rauf@corp.az',  3500, 20, 1,    '2018-06-15'),
  (3, 'Leyla', 29, NULL,            3000, 20, 2,    '2020-01-10'),
  (4, 'Orxan', 33, 'orxan@corp.az', 2800, 10, 1,    '2019-09-01'),
  (5, 'Nigar', 27, 'nigar@corp.az', 2800, 10, 4,    '2021-04-20'),
  (6, 'Vusal', 41, NULL,            4200, 30, 1,    '2016-11-05'),
  (7, 'Sara',  25, 'sara@corp.az',  2200, 30, 6,    '2022-02-14'),
  (8, 'Kamal', 52, 'kamal@corp.az', 6000, 40, 1,    '2012-08-30'),
  (9, 'Ilkin', 31, 'ilkin@corp.az', 3500, 40, 8,    '2020-07-07'),
  (10,'Anar',  36, 'anar@corp.az',  3000, NULL, 1,  '2019-05-12');

INSERT INTO contractors VALUES
  (1, 'Sara', 25.00), (2, 'Elnur', 30.00),
  (3, 'Ilkin', 28.50), (4, 'Tural', 22.00);
```

1. Draw ER diagram. Use Entity, Attributes, Relationships and Cardinality.

2. Create and change data (DDL and DML). Write the statements for:
    a. a table `leaves` with `leave_id` (auto-incrementing primary key), `emp_id` (integer, required, foreign key to `employees`), `start_date` (date, required) and `days` (integer, default 1);
    b. adding a column `is_active` (boolean, default true) to `employees`;
    c. inserting two new contractors, `(5, 'Murad', 27.50)` and `(6, 'Zaur', 31.00)`, with one INSERT statement;
    d. setting the email of the employee with `emp_id` 3 to `'leyla@corp.az'`;
    e. deleting all contractors whose `hourly_rate` is below 23.
3. Select, filter and sort.
    a. Return `name`, `hire_date` and `salary` of employees hired between 2019-01-01 and 2021-12-31 (both included) who earn less than 3500. Show the newest hire first and only the first 3 rows.
    b. Return `name` and `manager_id` of all employees, sorted by `manager_id` from the smallest to the largest; employees without a manager come last, and employees with the same manager are sorted by name.
    c. Return `name`, `email` and `salary` of employees who either have no email (NULL) or earn 5000 or more, highest salary first.
4. Joins.
    a. Return `name`, `salary` and `dept_name` of employees who work in a department located in Ganja or Sumgait. Show the highest salary first, equal salaries by name.

b. Return the `name` of every employee that has no department (use a LEFT JOIN and IS NULL, not a subquery).

c. Using a self join, return the `name` and `salary` of every employee who earns at least as much as his or her manager, together with `manager_name` and `manager_salary`; order by name.

5. Subqueries, CTE and set operations.

   a. Using a CTE named `high_earners` (employees who earn more than 3000), return the `dept_name` of every department that has at least 2 high earners, together with their number as `cnt`; order by dept_name.

   b. Using IN with a subquery, return `name` and `salary` of employees who work in a department located in Baku, highest salary first, equal salaries by name.

   c. Using a set operation, return all different names that appear in `employees` or in `contractors`, in alphabetical order.

6. Aggregation and window functions.

   (a) For employees that have a department, return `dept_id`, `headcount`, `lowest_salary` and `highest_salary` for each department, but only for departments with at least 2 employees and a lowest salary of 2800 or more; order by `highest_salary` from the largest to the smallest.

   (b) Return `name`, `hire_date`, `salary`, `hire_order` (the employee's position when all employees are numbered by `hire_date` from the oldest to the newest) and `dept_avg` (the average salary of the employee's department, rounded to 1 decimal) for all employees; order by hire_order.